Universidad de El Salvador

Facultad de Ingeniería y Arquitectura

Educación en Línea

Ingeniería en Desarrollo de Software

Asignatura:

Desarrollo y Reutilización de Software

Grupo Teórico:

Grupo Teórico 1

Proyecto – DRS135 – Documento de componentes e implementación

Integrantes:

Grupo 1 – Tema 5		
No	**Nombre completo**	**Correo institucional**
1	Alejandra Michelle Mejía Rivas	mr22035@ues.edu.sv
2	Christopher Alexis Velásquez Aguilar	va22020@ues.edu.sv
3	Iván Eduardo López Tobar	lt22009@ues.edu.sv
4	Josué Daniel Rodríguez Yanes	ry22001@ues.edu.sv
5	Irvin Elías Torres Merlos	tm22012@ues.edu.sv
6	Carlos Alberto Ávila González	ag22029@ues.edu.sv
7	José Emmanuel García Rodríguez	gr22081@ues.edu.sv

Fecha:

25/10/2026

Ingeniería en DESARROLLO DE SOFTWARE

COMPONENTES E IMPLEMENTACIÓN

1. Visión general

El proyecto es una API REST que administra usuarios. Cada usuario tiene tres datos (id, nombre y correo) y la API permite crearlos, listarlos, consultarlos, actualizarlos y eliminarlos. Estas cinco acciones se conocen como CRUD, por las siglas en inglés de Create, Read, Update y Delete.

Una API REST es una interfaz que se consume mediante el protocolo HTTP. Cada elemento que se administra (aquí, los usuarios) es un recurso identificado por una URL, en este caso /api/usuarios, y lo que se desea hacer con él se indica con el método HTTP: POST para crear, GET para consultar, PUT para actualizar y DELETE para eliminar. La información viaja en formato JSON y cada petición es independiente de las demás.

Los cuatro componentes

El código se divide en piezas pequeñas, cada una con una sola responsabilidad:

Clase	Paquete	Responsabilidad
Usuario	model	Describe los datos de un usuario.
GestorUsuarios	manager	Lógica de negocio: CRUD en memoria y persistencia en usuarios.json.
UsuarioController	controller	Capa de API REST: expone los endpoints HTTP.
ApiUsuariosApplication	raíz	Punto de entrada: arranca la aplicación y el servidor.

La idea central es que el controlador solo traduce entre HTTP y Java, el gestor concentra la lógica y el estado, y el modelo únicamente describe los datos. Gracias a esta separación, cada pieza se puede entender, probar y cambiar por separado.

2. Usuario: el modelo de datos

La clase Usuario es un POJO (Plain Old Java Object): una clase Java común, sin dependencia de frameworks ni lógica de negocio, cuyo único propósito es contener datos. Sus atributos (id, nombre, correo) son privados y se leen o modifican mediante métodos getter y setter, lo que aplica el principio de encapsulamiento.

Tres detalles tienen una razón técnica:

- Constructor vacío. Cuando llega un JSON, la librería Jackson primero crea un objeto vacío con este constructor y después asigna cada valor llamando a los setter. Sin el constructor vacío, la conversión fallaría.
- Getters. Al responder, Jackson hace el camino inverso: lee los valores con los getter para construir el JSON de salida.
- El atributo ID: No lo envía el cliente; lo asigna el gestor al crear el usuario.

3. GestorUsuarios: lógica y estado

GestorUsuarios es el núcleo de la aplicación. Guarda a los usuarios en memoria, genera sus identificadores, ofrece las operaciones del CRUD y mantiene una copia en el archivo usuarios.json para que los datos sobrevivan cuando la aplicación se detiene.

3.1 Patrón Singleton con Holder

Singleton es un patrón de diseño creacional que garantiza que una clase tenga una única instancia y ofrece un punto de acceso global a ella. Aquí es necesario porque el gestor contiene el estado de toda la aplicación: si cada petición creara su propio gestor, un usuario creado con POST no existiría para el siguiente GET.

Se implementa con tres elementos:

- Un constructor privado, para que ninguna otra clase pueda crear instancias con new,
- Una clase interna estática Holder, que guarda la única instancia en un atributo static final.

Un método público getInstance(), único punto de acceso, que siempre devuelve

esa instancia

```
private GestorUsuarios() {
   cargarDesdeArchivo();
}


private static class Holder {
        private static final GestorUsuarios INSTANCIA =
        new GestorUsuarios();
}


public static GestorUsuarios getInstance() {
```

```
        return Holder.INSTANCIA;
}
```

Esta variante se llama Initialization-on-Demand Holder. La clase Holder no se carga hasta que alguien invoca getInstance() por primera vez (inicialización perezosa). En ese momento la JVM inicializa Holder y crea la instancia, y la especificación de Java garantiza que la inicialización de una clase ocurre una sola vez y de forma segura, aunque haya varios hilos. Por eso no hace falta synchronized en getInstance() ni se paga costo de sincronización en cada acceso.

Como el constructor llama a cargarDesdeArchivo(), los datos guardados se leen una sola vez, la primera vez que se usa el gestor.

3.2 Almacenamiento en memoria

Los usuarios se guardan en un Map<Integer, Usuario> donde la llave es el id, lo que permite buscar un usuario sin recorrer toda la colección. La implementación elegida es ConcurrentSkipListMap, una colección del JDK con dos propiedades útiles:

- Segura para hilos (thread-safe). Un servidor web atiende varias peticiones al mismo tiempo, cada una en su propio hilo. Un HashMap común puede corromperse si dos hilos lo modifican a la vez; esta estructura está diseñada para ese escenario.
- Ordenada. Mantiene las llaves ordenadas (es una lista de saltos, o skip list), por lo que listar() devuelve los usuarios en orden de id.

3.3 Generación de identificadores

AtomicInteger es un contador entero cuyas operaciones son atómicas, es decir, indivisibles. El método getAndIncrement() devuelve el valor actual y lo incrementa en un solo paso, de modo que dos peticiones simultáneas nunca reciben el mismo id. Lo logra con instrucciones del procesador (compare-and-swap) en lugar de bloqueos, por lo que es muy rápido. El contador empieza en 1; si ya existe un archivo, se reajusta al cargarlo: el mayor id guardado más uno.

3.4 Operaciones del CRUD

Método	Qué hace	Devuelve
agregar	Toma el siguiente id, se lo asigna al usuario, lo guarda en el mapa y persiste en el archivo.	El usuario creado
listar	Copia los valores del mapa en un ArrayList nuevo.	Lista de usuarios
obtenerPorId	Busca el id en el mapa.	Optional<Usuario>
actualizar	Si existe, reemplaza nombre y correo (no el id) y persiste.	Optional<Usuario>
eliminar	Quita el usuario del mapa y, si existía, persiste.	boolean

listar() devuelve una copia y no el mapa interno: quien reciba la lista no puede agregar ni quitar elementos de la colección privada del gestor.

3.5 Optional

Optional<T> es un contenedor que puede tener un valor o estar vacío. Se usa en lugar de devolver null cuando un usuario no existe: obliga a quien llama a considerar el caso de ausencia y evita los NullPointerException. Además, deja la decisión de qué hacer con esa ausencia al controlador, que la traduce a un código HTTP 404; el gestor no necesita saber nada de HTTP.

3.6 Sincronización con synchronized

Las colecciones concurrentes garantizan que cada operación individual (un get, un put) sea segura, pero no que una secuencia de operaciones lo sea. Actualizar un usuario implica buscarlo, modificar dos campos y escribir el archivo; si otro hilo lo eliminara en medio de esa secuencia, el resultado sería inconsistente.

La palabra reservada synchronized en actualizar y eliminar hace que solo un hilo a la vez pueda ejecutar esos métodos sobre la instancia (todo objeto Java tiene un candado, o monitor). Así se protege la secuencia completa de leer, modificar y escribir. Las lecturas (listar, obtenerPorId) no se bloquean, de modo que no pierden rendimiento.

3.7 Persistencia en JSON

Persistir es guardar los datos de modo que sobrevivan al apagado del programa. Aquí se usa un archivo de texto, usuarios.json, en lugar de una base de datos, por simplicidad. La conversión entre objetos Java y texto JSON la hace ObjectMapper, de Jackson:

Serialización: objeto Java a JSON (writeValue).

Deserialización: JSON a objeto Java (readValue).

La opción INDENT_OUTPUT agrega sangría al JSON para que sea legible.

cargarDesdeArchivo() se ejecuta una sola vez, al crearse el Singleton. Si el archivo no existe (primera ejecución) no hace nada. Si existe, lo lee como una List<Usuario>, llena el mapa y recalcula el siguiente id. Se usa TypeReference porque Java borra la información de los tipos genéricos en tiempo de ejecución (type erasure); sin ella, Jackson no sabría que debe construir una lista de Usuario.

guardarEnArchivo() se invoca después de cada cambio y reescribe el archivo completo con la lista actual. Es una estrategia sencilla, aunque su costo crece con la cantidad de usuarios; para un volumen grande convendría una base de datos. Si ocurre un error de lectura o escritura, se muestra un mensaje en la consola de errores y la aplicación sigue funcionando. Como la ruta es relativa, el archivo se crea en la carpeta desde la que se ejecuta la aplicación (la raíz del proyecto).

4. UsuarioController: la capa de API

El controlador recibe las peticiones HTTP, las traduce en llamadas al gestor y convierte el resultado en una respuesta HTTP con el código de estado adecuado. No contiene lógica de negocio. Obtiene el gestor con GestorUsuarios.getInstance(), de modo que siempre trabaja con la misma instancia.

4.1 Anotaciones utilizadas

Anotación	Función
@RestController	Marca la clase como controlador REST. Combina @Controller y @ResponseBody: lo que devuelve cada método se convierte a JSON y se escribe directamente en el cuerpo de la respuesta.
@RequestMapping	Define la ruta base de todos los endpoints de la clase; aquí, /api/usuarios.

@PostMapping, @GetMapping, @PutMapping, @DeleteMapping	Asocian un método Java con un verbo HTTP y, si se indica, una ruta adicional como "/{id}".
@RequestBody	Toma el cuerpo JSON de la petición y lo convierte en un objeto Usuario mediante Jackson.
@PathVariable	Extrae un valor de la URL ({id}) y lo entrega como parámetro del método.

4.2 ResponseEntity y códigos de estado

ResponseEntity<T> representa una respuesta HTTP completa: código de estado, encabezados y cuerpo. Permite elegir el código en cada caso:

Método Java	**Petición**	**Éxito**	**Si no existe el id**
agregar	POST /api/usuarios	201 Created, con el usuario creado	No aplica
listar	GET /api/usuarios	200 OK	No aplica
obtener	GET /api/usuarios/{id}	200 OK, con el usuario	404 Not Found
actualizar	PUT /api/usuarios/{id}	200 OK, con el usuario	404 Not Found
eliminar	DELETE /api/usuarios/{id}	204 No Content, sin cuerpo	404 Not Found

listar() devuelve la lista directamente: cuando un método no usa ResponseEntity, Spring responde 200 OK por defecto.

4.3 Manejo del Optional

Como obtenerPorId y actualizar devuelven un Optional, el controlador usa un estilo funcional:

```
return gestor.obtenerPorId(id)

    .map(ResponseEntity::ok)

    .orElse(ResponseEntity.notFound().build());
```

Si el Optional contiene un usuario, map lo envuelve en una respuesta 200 con el usuario en el cuerpo; si está vacío, orElse produce una respuesta 404. En eliminar se usa un operador ternario sobre el boolean: 204 si se eliminó, 404 si no existía. En actualizar, el id que manda es el de la URL; del JSON recibido solo se copian nombre y correo.

5. ApiUsuariosApplication y Spring Boot

Esta clase contiene el método main, el punto de entrada de todo programa Java. Su única instrucción, SpringApplication.run(...), pone en marcha toda la aplicación.

@SpringBootApplication es una anotación compuesta que equivale a tres:

- @SpringBootConfiguration: marca la clase como fuente de configuración.
- @EnableAutoConfiguration: Spring Boot revisa las dependencias disponibles y configura automáticamente lo necesario. Al encontrar el starter web, prepara el servidor, el enrutamiento y la conversión JSON.
- @ComponentScan: busca clases anotadas (como @RestController) en el paquete de la clase principal y sus subpaquetes. Por eso el controlador se detecta sin registrarlo manualmente.

Al ejecutarse run, Spring realiza estos pasos:

1. Crea el contenedor de Inversión de Control (IoC), que construye y administra los objetos de la aplicación (beans).
2. Aplica la autoconfiguración y el escaneo de componentes.
3. Inicia Tomcat embebido, un servidor web que vive dentro de la propia aplicación, por defecto en el puerto 8080. No hace falta instalar un servidor aparte.
4. Registra el DispatcherServlet.

El DispatcherServlet implementa el patrón Front Controller: es la puerta de entrada de todas las peticiones. Revisa la URL y el método HTTP y decide a qué método del controlador entregarla.

6. Patrones de diseño y principios

6.1 Singleton

Se aplica en GestorUsuarios, en su variante Initialization-on-Demand Holder. Su propósito es que todos los endpoints trabajen sobre el mismo estado en memoria. El detalle de su implementación está en la sección 3.1.

6.2 MVC (Modelo, Vista, Controlador)

MVC divide una aplicación según tres responsabilidades. En esta API se reparte así:

Componente MVC	En el proyecto
Modelo	Usuario, que representa los datos del dominio.
Controlador	UsuarioController, que atiende las peticiones HTTP y delega.
Vista	La representación JSON de la respuesta, generada por Jackson.

El GestorUsuarios actúa como capa de servicio entre el controlador y los datos.

6.3 Separación de responsabilidades

Cada clase tiene una sola razón para cambiar. Si el archivo JSON se reemplazará por una base de datos, solo habría que modificar GestorUsuarios; el controlador y el modelo seguirán igual.

Una observación: Spring crea sus beans como singleton por defecto, así que un resultado equivalente se lograría con la anotación @Service. Implementar el patrón manualmente permite ver cómo funciona por dentro.

7. Recorrido de una petición

Ejemplo: crear un usuario con POST /api/usuarios y el cuerpo {"nombre": "Ana", "correo": "ana@ues.edu.sv"}.

1. El cliente (por ejemplo, Thunder Client) envía la petición. Tomcat la recibe y se la pasa al DispatcherServlet.
2. El DispatcherServlet busca el método asociado a POST en /api/usuarios: agregar.
3. Por @RequestBody, Jackson crea un Usuario vacío y le asigna nombre y correo con los setter.
4. El controlador llama a gestor.agregar(usuario); getInstance() entrega siempre la misma instancia.
5. El gestor obtiene el id con getAndIncrement(), lo asigna, guarda el usuario en el mapa y reescribe usuarios.json.
6. El controlador envuelve el usuario en un ResponseEntity con código 201 Created. Jackson lo convierte a JSON y la respuesta vuelve al cliente.

Una consulta como GET /api/usuarios/7 sigue el mismo camino: @PathVariable entrega el valor 7, el gestor lo busca en el mapa y el controlador responde 200 con el usuario, o 404 si el Optional viene vacío.